Created by data-to-paper (AI)

Revealing System Behavior through Event-Log Associations in OpenStack

data-to-paper

April 8, 2026

Abstract

System logs play a critical role in cloud computing environments by capturing significant events and their associated log levels, which are crucial for maintaining system reliability and optimizing performance. While extensive research has focused on log analysis for fault detection, the specific interactions between event identifiers (EventIds) and log levels in OpenStack logs remain underexamined. This study addresses this gap by analyzing associations between EventIds and log levels in a structured dataset of 2000 OpenStack log entries. Utilizing chi-squared analyses, the study uncovers significant non-random associations, highlighting that specific components, like the 'Libvirt Image Cache,' exhibit strong linkages between EventIds and logging behaviors. These findings suggest that particular patterns in log levels are indicative of component-specific behaviors, offering valuable insights for tailoring log management strategies. Despite the enlightening findings, the results are constrained by the dataset's specificity, advising caution against overgeneralization. The outcomes underscore the potential for developing enhanced diagnostic frameworks and log level strategies to improve cloud infrastructure resilience.

Introduction

In the rapidly evolving landscape of cloud computing, system logs serve an indispensable role in managing and optimizing cloud infrastructures by recording critical operational events [1, 2]. These logs provide valuable insights that can drive critical decision-making processes and enhance system reliability, particularly in complex environments like OpenStack [3]. As such, understanding the interaction between event identifiers (EventIds) and log levels within these logs becomes crucial for maintaining robust and efficient cloud systems [4, 5].

Created by data-to-paper (AI)

Despite considerable advances in log-based fault detection and analysis, existing literature has primarily focused on broader log management strategies or specific case studies, often overlooking the nuanced relationships between distinct log attributes within cloud frameworks [6]. For instance, while vulnerability patterns in cloud audit logs have been explored [7], there remains a lack of in-depth analysis concerning how the distribution of EventIds correlates with different log levels across cloud components. This research gap highlights the need for a more targeted examination of log events to uncover how these dynamics influence system diagnostics and maintenance [8].

In response to this gap, our study investigates the correlation between EventIds and log levels within a structured dataset of OpenStack logs [9]. By leveraging statistical analysis methods to assess these associations, the study offers new insights into component-specific logging behavior that could inform more effective log management approaches [10, 11]. The findings aim to contribute to the refinement of diagnostic techniques and the enhancement of cloud infrastructure resilience [12].

To achieve these aims, the study employs a chi-squared statistical test to explore associations within the dataset, providing a rigorous framework for data analysis [13]. Our analysis explores the frequency of EventIds at various log levels, revealing significant non-random associations that underline distinct logging behaviors related to specific system components. This methodological approach not only strengthens the reliability of our findings but also sets the stage for future research aimed at optimizing log strategies in cloud computing environments [14, 13].

Results

Initially, to understand the frequency distribution of Event IDs and their association with different log levels, we conducted an analysis summarizing the most common Event IDs at the 'INFO' and 'WARNING' levels. The bar plot in Figure 1 delineates the frequency of top Event IDs across these log levels on a logarithmic scale. The analysis focused on event patterns that were indexed and frequently observed, such as Event 25 and Event 34, with considerable occurrences 931 and 82, respectively, predominantly reported at the 'INFO' level. This characterization provides a foundational understanding of which event identifiers predominantly feature at lower log levels.

Subsequently, a chi-squared test was executed to evaluate the overall as-

Created by data-to-paper (AI)

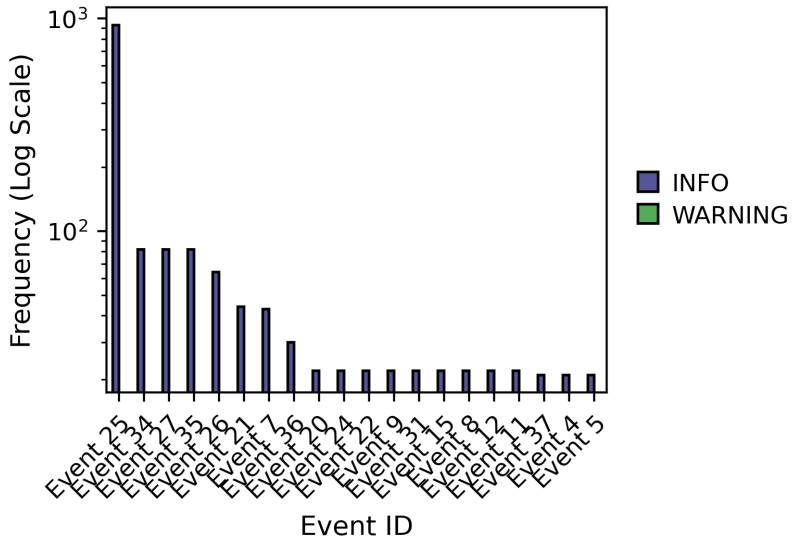

Figure 1: Frequency of Top Event IDs Across Log Levels INFO: Informational log level. WARNING: Warning log level. Event 25: Specific event pattern indexed as E25. Event 34: Specific event pattern indexed as E34. Event 27: Specific event pattern indexed as E27. Event 35: Specific event pattern indexed as E35. Event 26: Specific event pattern indexed as E26. Event 21: Specific event pattern indexed as E21. Event 7: Specific event pattern indexed as E7. Event 36: Specific event pattern indexed as E36. Event 20: Specific event pattern indexed as E20. Event 24: Specific event pattern indexed as E24. Event 22: Specific event pattern indexed as E22. Event 9: Specific event pattern indexed as E9. Event 31: Specific event pattern indexed as E31. Event 15: Specific event pattern indexed as E15. Event 8: Specific event pattern indexed as E8. Event 12: Specific event pattern indexed as E12. Event 11: Specific event pattern indexed as E11. Event 37: Specific event pattern indexed as E37. Event 4: Specific event pattern indexed as E4. Event 5: Specific event pattern indexed as E5.

Created by data-to-paper (AI)

sociation between Event IDs and log levels. The results presented in Table 1 show a significant chi-squared statistic of 133.8 with 20 degrees of freedom, indicating a highly significant relationship (P-value $< 10^{-6}$). This statistical insight confirms that there is a non-random association between Event IDs and the levels of log messages, suggesting a distinct pattern in logging behaviors across different event identifiers.

Table 1: Overall Chi-Squared Test for Event ID and Log Level Association

	ChiSq	PVal	DF	Sig.
EventId Level Test	133.8	$<10^{-6}$	20	***

ChiSq: Chi-Square statistic for test of independence
PVal: Calculated probability for the Chi-Square statistic
DF: Degrees of freedom for the Chi-Square test
Sig.: Statistical significance notation: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, ns $p >= 0.05$
EventId Level Test: Chi-Square test for association between EventId and Log Level

To gain a more granular understanding, we further stratified the chi-squared analysis by components to ascertain whether specific components exhibit stronger associations between Event IDs and log levels. Table 2 presents these stratified results, revealing that certain components, such as the 'Libvirt Image Cache', have a strong statistical association, evidenced by a chi-squared statistic of 250.9 with statistically significant P-value (P-value $< 10^{-6}$). In contrast, other components featured less robust or indeterminate associations, either due to insufficient data or lack of variability in log levels, evidenced by '-' indicators in several instances.

In summary, these results demonstrate significant associations between Event IDs and log levels within OpenStack system logs, particularly pronounced in certain components. This suggests distinct logging behaviors and potential areas for optimizing log level assignments to enhance system diagnostic capabilities.

Discussion

This study focused on the exploration of associations between event identifiers (EventIds) and log levels within OpenStack logs, a critical aspect of cloud infrastructure management that has been relatively underexplored compared to other log analysis domains [1, 2]. Previous investigations have largely concentrated on different facets of log management and fault detec-

Created by data-to-paper (AI)

Table 2: Chi-Squared Statistics for Association Between Event ID and Log Level Stratified by Component

Component	ChiSq	PVal	DF	Sig.
Compute WSGI	-	-	-	-
Compute Manager	3.042	0.881	7	ns
Libvirt Image Cache	250.9	$<10^{-6}$	5	***
Compute Server Ext. Events	-	-	-	-
Libvirt Driver	-	-	-	-
Resource Tracker	-	-	-	-
Metadata WSGI	-	-	-	-
API WSGI	-	-	-	-
Compute Claims	-	-	-	-
Scheduler Host Manager	-	-	-	-

ChiSq: Chi-Square statistic for test of independence
PVal: Calculated probability for the Chi-Square statistic
DF: Degrees of freedom for the Chi-Square test
Sig.: Statistical significance notation: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, ns $p >= 0.05$
Compute WSGI: Web Server Gateway Interface in OpenStack Compute Module
Metadata WSGI: Web Server Gateway Interface for Metadata Service in OpenStack
API WSGI: Web Server Gateway Interface for API in OpenStack

tion in cloud environments, with varying emphasis on log-based diagnosis [15, 16], but there remains an open question regarding the specific interactions between EventIds and log levels.

Our methodological approach involved a chi-squared statistical analysis of a structured dataset consisting of 2000 OpenStack log entries. This dataset captured valuable system event information categorized into different log levels, particularly 'INFO' and 'WARNING'. The primary aim was to identify non-random associations between EventIds and these log levels, shedding light on log behavior patterns that could potentially inform optimized log management strategies [8, 12].

Our analysis revealed significant associations, as highlighted by a chi-squared statistic of 133.8 with 20 degrees of freedom, indicating a markedly significant non-random association (P-value $< 10^{-6}$). Moreover, our component-wise analysis revealed that specific components, notably the 'Libvirt Image Cache', demonstrated pronounced associations, with a chi-squared statistic of 250.9 [6]. This aligns with previous findings suggesting component-specific

Created by data-to-paper (AI)

logging behaviors and supports the notion that understanding these patterns can enhance system diagnostics [7].

However, this study does present certain limitations. The analysis was limited to a dataset from a single OpenStack instance, which may not fully capture the variability present in more extensive deployments. The dataset size, while manageable, may also restrict the generalization of findings to larger cloud systems. Additionally, the study's focus on two log levels may have overlooked nuances present in higher-severity log levels, such as 'ERROR' or 'CRITICAL', which could provide further insights [17].

In conclusion, this study successfully identified significant associations between EventIds and log levels within OpenStack logs, particularly emphasizing component-specific behaviors that could enhance system diagnostic frameworks. The findings offer valuable implications for developing targeted log management strategies that address the distinct needs of cloud environments. Moving forward, expanding the dataset to include various OpenStack deployments and incorporating a broader range of log levels could provide a more comprehensive understanding. Additionally, integrating machine learning techniques for pattern recognition may further advance cloud log analysis capabilities, offering refined insights into system performance and reliability [3].

Methods

Data Source

The data utilized in this study originates from a single structured log file containing 2000 OpenStack log entries. This dataset captures valuable system information in the form of log events identified by unique EventIds, along with corresponding log levels categorized into various states such as INFO and WARNING. The dataset serves as a representative sample, specifically tailored for the research goal of exploring the correlation between log event frequencies and types, and the operational states of the system as indicated by different log levels within the OpenStack environment.

Data Preprocessing

In the context of this study, no explicit data preprocessing procedures such as cleaning or transformation were carried out, as the dataset was already structured in a manner conducive to our analysis. The analysis centered on

Created by data-to-paper (AI)

utilizing categorical data, particularly focusing on EventIds and log levels without the need for additional preprocessing.

Data Analysis

The analysis commenced with the descriptive examination of event frequencies and the corresponding log levels. This involved computing the frequency of occurrences for various EventIds, stratified by log levels, to identify predominant events in the dataset. A chi-squared statistical test was employed to assess the association between EventIds and log levels, thus determining the significance of non-random patterns in log event distribution. The dataset was further stratified based on specific components to perform component-wise chi-squared tests, examining the interactions between EventIds and log levels within different system components. This analysis was performed to understand if particular components exhibited distinctive log behavior patterns. The results of these tests were subsequently presented in the form of tables, highlighting the chi-squared statistics, p-values, and degrees of freedom, with significance assessed according to established thresholds.

Code Availability

Custom code used to perform the data preprocessing and analysis, as well as the raw code outputs, are provided in Supplementary Methods.

References

[1] Manish Kumar and M. Hanumanthappa. Scalable intrusion detection systems log analysis using cloud computing infrastructure. 2013.

[2] Yue Yuan, Wenchang Shi, Bin Liang, and Bo Qin. An approach to cloud execution failure diagnosis based on exception logs in openstack. 2019.

[3] Janit Rajkarnikar, Nishan Poudel, and Nick Rahimi. Unsupervised anomaly detection in openstack logs via fine-tuned roberta embeddings. *Journal of Cybersecurity, Digital Forensics and Jurisprudence*, 1, 2025.

[4] Sa Meng. Reliability simulation in cloud computing system. *International Journal of Performability Engineering*, 2018.

Created by data-to-paper (AI)

[5] Dalal Alharthi and Rozhin Yasaei. Llm-powered automated cloud forensics: From log analysis to investigation. 2025.

[6] Patrick Kubiak, Stefan Rass, and Martin Pinzger. It-application behaviour analysis: Predicting critical system states on openstack using monitoring performance data and log files. 2020.

[7] Rui Xie, Rose Gamble, and Norman Ahmed. *Diagnosing Vulnerability Patterns in Cloud Audit Logs*. Springer New York, 2013.

[8] De-Qing Zou, Hao Qin, and Hai Jin. Uilog: Improving log-based fault diagnosis by log analysis. *Journal of Computer Science and Technology*, 31(5), 2016.

[9] Vaibhav Agrawal, Devanjal Kotia, Kamelia Moshirian, and Mihui Kim. Log-based cloud monitoring system for openstack. 2018.

[10] Uchit Vyas. *OpenStack Deployment*. Apress, 2016.

[11] Takayuki Kushida. Distributed log management for dynamically changing computing environments on cloud. *International Journal of Big Data Intelligence*, 7(3), 2020.

[12] Robert Andrews and Moe Thandar Wynn. Event log imperfection patterns for process mining: Towards a systematic approach to cleaning event logs. *Information Systems*, 137, 2026.

[13] *A chi squared test*. Cambridge University Press, 2022.

[14] *Categorical Data Analysis*. SAGE Publications, Inc., 2009.

[15] Jo Swinnen, Koen Vanhoof, and Els Hannes. Querying event logs: Discovering non-events in event logs. 2010.

[16] Bo Feng, Chentao Wu, and Jie Li. Mlc: An efficient multi-level log compression method for cloud backup systems. 2016.

[17] Matthew Akanle, Emmanuel Adetiba, Victor Akande, Adekunle Akinrinmade, Sunday Ajala, Funmilayo Moninuola, Joke Badejo, and Ezekiel Adebiyi. Experimentations with openstack system logs and support vector machine for an anomaly detection model in a private cloud infrastructure. 2020.

Created by data-to-paper (AI)

A Data Description

Here is the data description, as provided by the user:

```
## General Description
Dataset for FARS proposal FA0408 from https://analemma.ai/fars/.
    File: OpenStack_2k.log_structured.csv

## Data Files
The dataset consists of 1 data file:

### "OpenStack_2k.log_structured.csv"
Data file for FA0408. File: OpenStack_2k.log_structured.csv.
    Explore columns to find statistically testable hypotheses.
```

B Data Exploration

B.1 Code

The Data Exploration was carried out using the following custom code:

```
import pandas as pd

# Load the data
df = pd.read_csv("OpenStack_2k.log_structured.csv")

# Open output file
with open("data_exploration.txt", "w") as file:

    # Data Size
    file.write("# Data Size\n")
    file.write(f"Number of rows: {df.shape[0]}\n")
    file.write(f"Number of columns: {df.shape[1]}\n\n")

    # Summary Statistics
    file.write("# Summary Statistics\n")
    file.write(df.describe(include='all').to_string() + "\n\n")

    # Categorical Variables
    file.write("# Categorical Variables\n")
    categorical_cols = df.select_dtypes(include=['object', '
        ↪ category']).columns
    if categorical_cols.empty:
        file.write("# Not Applicable\n")
    else:
        for col in categorical_cols:
            file.write(f"Column: {col}\n")
```

Created by data-to-paper (AI)

```
        file.write(f"Unique values: {df[col].nunique()}\n")
        most_common = df[col].mode().values
        file.write(f"Most common value(s): {', '.join(
            ↪ most_common)}\n\n")

# Missing Values
file.write("# Missing Values\n")
missing_values = df.isnull().sum()
if missing_values.sum() == 0:
    file.write("# Not Applicable\n")
else:
    for col, count in missing_values.items():
        if count > 0:
            file.write(f"Column: {col} - Missing Values: {
                ↪ count}\n")
file.write("\n")

# Other potential summary: Log Levels
file.write("# Log Level Distribution\n")
if 'Level' in df.columns:
    level_counts = df['Level'].value_counts()
    file.write(level_counts.to_string() + "\n\n")
else:
    file.write("# Not Applicable\n\n")

# Other potential summary: Event IDs
file.write("# Event ID Distribution\n")
if 'EventId' in df.columns:
    event_counts = df['EventId'].value_counts()
    file.write(event_counts.to_string() + "\n\n")
else:
    file.write("# Not Applicable\n\n")
```

B.2 Code Description

The data exploration process was implemented to gain insights into the dataset represented by the file "OpenStack_2k.log_structured.csv". This process was systematically conducted through the following steps:

B.3 Data Size Assessment

The first step in the exploration was to determine the dimensions of the dataset. This includes finding the number of rows and columns to understand the scale of data being worked with. This information is essential as it informs the complexity and potential processing time required for further analysis.

Created by data-to-paper (AI)

B.4 Summary Statistics Calculation

Next, summary statistics were generated for all the columns in the dataset. This includes descriptive metrics such as count, unique values, top frequency items, mean, standard deviation, minimum, and maximum values. By including all data types, the statistics provided an overview of the data distribution, central tendencies, and variability within the dataset, informing any normalization or transformation needs.

B.5 Categorical Variables Exploration

The exploration process then identified and examined categorical variables within the dataset. For each categorical column, the goal was to find the uniqueness and frequency of the values they contained. Identifying the most common value(s) can explore potential biases and trends within these categorical data, which might suggest areas for more detailed analysis.

B.6 Missing Values Analysis

An analysis of missing values was also conducted. Understanding the extent and pattern of missing data is crucial, as it can adversely impact the integrity and validity of statistical analysis or machine learning models. The code reported columns with missing data and quantified the extent of these missing values, guiding decisions on data cleaning or imputation methods.

B.7 Log Level Distribution

As the dataset appears to pertain to event logging, an analysis of log level distribution was conducted. This involved examining the frequency distribution of log level entries, which aids in understanding the severity and prevalence of different types of events within the dataset. This information can be valuable for identifying potential irregularities or areas of concern in system operations.

B.8 Event ID Distribution

Similarly, the distribution of unique Event IDs was analyzed. Analyzing the frequency and type of different events captured by the Event IDs offers insights into the dataset's coverage of system events and potential correlations with system performance or issues.

Overall, this structured exploration established a foundational understanding of the dataset's size, content, and quality, providing a framework for subsequent detailed statistical analysis or machine learning applications.

B.9 Code Output

data_exploration.txt

```
# Data Size
Number of rows: 2000
Number of columns: 11

# Summary Statistics
       LineId                            Logrecord        Date
                    Time       Pid Level
           Component
                                                          ADDR

           Content EventId
           EventTemplate
count    2000                                 2000        2000
             2000      2000  2000                              2000
                                                     2000

    2000    2000                                                 2000
unique    NaN                                     3           1
             1933       NaN     2                                10
                                                      939

    1548      43                                                  43
top       NaN  nova-api.log.1.2017-05-16_13:53:08  2017-05-16
    00:05:21.242       NaN  INFO  nova.osapi_compute.wsgi.server
    req-addc1839-2ed5-4778-b57e-5854eb7b8b09 - - - - -  image 0673
    dd71-34c5-4fbb-86c4-40623fbe45b4 at (/var/lib/nova/instances/
    _base/a489c868f0c37da93b76227c91bb03908ac0e742): in use: on
    this node 1 local, 0 on other nodes sharing this instance
    storage     E25  <*> "GET <*>" status: <*> len: <*> time:
    <*>.<*>
freq      NaN                                 1060        2000
                3       NaN  1969                                809
                                                      398

    82     931                                                     931
mean     1000                                   NaN          NaN
              NaN 1.511e+04   NaN                                  NaN
                                                      NaN

    NaN     NaN                                                     NaN
```


Created by data-to-paper (AI)

Created by data-to-paper (AI)

```
std     577.5                                   NaN          NaN
              NaN 1.139e+04   NaN                                  NaN
                                                      NaN

    NaN     NaN                                                    NaN
min         1                                   NaN          NaN
              NaN      2931   NaN                                  NaN
                                                      NaN

    NaN     NaN                                                    NaN
25%     500.8                                   NaN          NaN
              NaN      2931   NaN                                  NaN
                                                      NaN

    NaN     NaN                                                    NaN
50%      1000                                   NaN          NaN
              NaN     25746   NaN                                  NaN
                                                      NaN

    NaN     NaN                                                    NaN
75%      1500                                   NaN          NaN
              NaN     25746   NaN                                  NaN
                                                      NaN

    NaN     NaN                                                    NaN
max      2000                                   NaN          NaN
              NaN     25998   NaN                                  NaN
                                                      NaN

    NaN     NaN                                                    NaN

# Categorical Variables
Column: Logrecord
Unique values: 3
Most common value(s): nova-api.log.1.2017-05-16_13:53:08

Column: Date
Unique values: 1
Most common value(s): 2017-05-16

Column: Time
Unique values: 1933
Most common value(s): 00:05:21.242, 00:06:43.298, 00:11:33.093,
    00:12:05.112

Column: Level
Unique values: 2
Most common value(s): INFO
```

Created by data-to-paper (AI)

```
Column: Component
Unique values: 10
Most common value(s): nova.osapi_compute.wsgi.server

Column: ADDR
Unique values: 939
Most common value(s): req-addc1839-2ed5-4778-b57e-5854eb7b8b09 - -
     - - -

Column: Content
Unique values: 1548
Most common value(s): Active base files: /var/lib/nova/instances/
    _base/a489c868f0c37da93b76227c91bb03908ac0e742, image 0673dd71
    -34c5-4fbb-86c4-40623fbe45b4 at (/var/lib/nova/instances/_base
    /a489c868f0c37da93b76227c91bb03908ac0e742): checking, image
    0673dd71-34c5-4fbb-86c4-40623fbe45b4 at (/var/lib/nova/
    instances/_base/a489c868f0c37da93b76227c91bb03908ac0e742): in
    use: on this node 1 local, 0 on other nodes sharing this
    instance storage

Column: EventId
Unique values: 43
Most common value(s): E25

Column: EventTemplate
Unique values: 43
Most common value(s): <*> "GET <*>" status: <*> len: <*> time:
    <*>.<*>

# Missing Values
# Not Applicable

# Log Level Distribution
Level
INFO       1969
WARNING      31

# Event ID Distribution
EventId
E25    931
E34     82
E35     82
E27     82
E26     64
E21     44
E7      43
E36     30
E42     30
E8      22
```

Created by data-to-paper (AI)

```
E11      22
E24      22
E9       22
E12      22
E15      22
E22      22
E31      22
E20      22
E10      21
E16      21
E6       21
E3       21
E18      21
E19      21
E2       21
E1       21
E23      21
E17      21
E5       21
E33      21
E37      21
E13      21
E14      21
E4       21
E30      15
E32      15
E41      15
E28      15
E29       9
E40       5
E39       2
E38       1
E43       1
```

C Data Analysis

C.1 Code

The Data Analysis was carried out using the following custom code:

```python
# IMPORT
from my_utils import df_to_latex, df_to_figure
import pandas as pd
import numpy as np
from scipy.stats import chi2_contingency
import pickle

# LOAD DATA
```

Created by data-to-paper (AI)

```
file_path = 'OpenStack_2k.log_structured.csv'
data = pd.read_csv(file_path)

# DATASET PREPARATIONS
# No dataset preparations are needed.

# DESCRIPTIVE STATISTICS
## Figure df_desc_stat:
caption = "EventId frequency and associated log levels"
# Summarizing EventId and Level
eventid_level_counts = data.groupby(['EventId', 'Level']).size().
    ↪ unstack(fill_value=0)
eventid_level_counts = eventid_level_counts.sort_values(by='INFO',
    ↪  ascending=False).head(20)

df_to_figure(
    eventid_level_counts,
    'df_desc_stat',
    caption=caption,
    kind='bar',
    y=['INFO', 'WARNING'],
    logy=True
)

# PREPROCESSING
# No preprocessing is needed, because our analysis primarily
    ↪ relies on categorical data

# ANALYSIS
## Table df_eventid_level:
caption = "Chi-squared test for association between EventId and
    ↪ Log Level"
# Aggregating rare EventIds into 'Other'
top_event_ids = list(eventid_level_counts.index)
data['EventId_Aggregated'] = data['EventId'].apply(lambda x: x if
    ↪ x in top_event_ids else 'Other')
contingency_table = pd.crosstab(data['EventId_Aggregated'], data['
    ↪ Level'])
chi2_results = chi2_contingency(contingency_table)

# Create the dataframe for the table
df_eventid_level = pd.DataFrame({
    'Chi-Square': [chi2_results.statistic],
    'p-value': [chi2_results.pvalue],
    'dof': [chi2_results.dof],
    'Significance': ['***' if chi2_results.pvalue < 0.001 else '**
        ↪ ' if chi2_results.pvalue < 0.01 else '*' if chi2_results
        ↪ .pvalue < 0.05 else 'ns']
}, index=['EventId_Level_Test'])
```

Created by data-to-paper (AI)

```
df_to_latex(df_eventid_level, 'df_eventid_level', caption=caption)

## Table df_component_eventid_level:
caption = "Chi-squared test stratified by Component for EventId
    ↪ and Log Level"
# Stratifying by Component
results = []
for component in data['Component'].unique():
    component_data = data[data['Component'] == component]
    contingency_table = pd.crosstab(component_data['
        ↪ EventId_Aggregated'], component_data['Level'])
    if contingency_table.shape[1] > 1 and contingency_table.shape
        ↪ [0] > 1:
        chi2_results = chi2_contingency(contingency_table)
        significance = '***' if chi2_results.pvalue < 0.001 else '
            ↪ **' if chi2_results.pvalue < 0.01 else '*' if
            ↪ chi2_results.pvalue < 0.05 else 'ns'
    else:
        chi2_results = {'statistic': '-', 'pvalue': '-', 'dof': '-
            ↪ ',}
        significance = '-'

    results.append([
        component,
        chi2_results['statistic'],
        chi2_results['pvalue'],
        chi2_results['dof'],
        significance
    ])

# Create the dataframe for the table
df_component_eventid_level = pd.DataFrame(results, columns=['
    ↪ Component', 'Chi-Square', 'p-value', 'dof', 'Significance'])
df_component_eventid_level.set_index('Component', inplace=True)
df_to_latex(df_component_eventid_level, '
    ↪ df_component_eventid_level', caption=caption)

# SAVE ADDITIONAL RESULTS
additional_results = {
    'Total number of observations': len(data),
    'Chi-square statistic for EventId and Log Level': chi2_results
        ↪ ['statistic'],
    'p-value for EventId and Log Level': chi2_results['pvalue'],
    'dof for EventId and Log Level': chi2_results['dof'],
    # For stratified results, only save examples or summaries;
        ↪ avoid saving entire DataFrame
    'Number of components analyzed': len(
        ↪ df_component_eventid_level)
}
```

Created by data-to-paper (AI)

```
with open('additional_results.pkl', 'wb') as f:
    pickle.dump(additional_results, f)
```

C.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
        filename: str, caption: str,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    """


def df_to_figure(
        df, filename: str, caption: str,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
         ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
         ↪ plotted above the error bars.
```

Created by data-to-paper (AI)

```
If provided, the length of `y_ci`, and `y_p_value` should be
    ↪ the same as of `y`.

Example:
Suppose, we have:

df_lin_reg_longevity = pd.DataFrame({
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
        ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
        ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
        ↪ [0.0001, ...],
}, index=['var1', ...])

then:
df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
    ↪ caption='Coefficients of ...', kind='bar',
    y=['adjusted_coef', 'unadjusted_coef'],
    y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
    y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
"""
```

C.3 Code Description

C.4 Import and Load Data

The analysis begins by importing necessary libraries for data manipulation and statistical testing, specifically, pandas, numpy, and scipy.stats. The dataset ‘OpenStack_2k.log_structured.csv‘ is loaded into a pandas DataFrame, preparing it for subsequent analyses.

C.5 Descriptive Statistics

The purpose of this step is to visualize the frequency of different log levels within each EventId, providing an overview of how often each event and log level occurs. The dataset is grouped by ‘EventId‘ and ‘Level‘, counting occurrences for each combination, and the results are presented in a bar chart. This descriptive visual aids in identifying the most frequent EventIds, focusing initially on the top 20 for clarity.

C.6 Preprocessing

Given that the analysis is focused on categorical data, no particular preprocessing steps were necessary to transform or cleanse the data before con-

Created by data-to-paper (AI)

ducting statistical tests.

C.7 Chi-Squared Test of Independence

The analysis investigates whether there is an association between ‘EventId‘ and ‘Log Level‘ by using the Chi-squared test of independence. To facilitate this, rare EventIds are aggregated into a category labeled ’Other’ to avoid sparsity issues that can affect the reliability of the chi-squared test. A contingency table is created, which cross-tabulates ‘EventId_Aggregated‘ by ‘Level‘, and the chi-squared test is conducted. The results, including the chi-square statistic, p-value, degrees of freedom, and significance level, are compiled into a summary table.

C.8 Stratified Chi-Squared Test by Component

Further analysis is performed by stratifying the data by ‘Component‘ to examine whether the association between ‘EventId‘ and ‘Log Level‘ varies across different components of the system. For each unique component, a chi-squared test is performed if the contingency table is sufficiently populated (i.e., more than one level and one event). The results for each component, including test statistics and significance, are summarized in another table.

C.9 Result Storage

Finally, key results and summaries are stored using the ‘pickle‘ module to facilitate future reference and review. These results include the total number of observations, statistics concerning the chi-squared tests for both overall and stratified analyses, and the count of components analyzed.

This high-level structured approach ensures both comprehensive data exploration and rigorous statistical assessment of the relationships within the log data, particularly focusing on the independence between EventId and log levels across different system components.

C.10 Code Output

df_component_eventid_level.pkl

```
                                                    Chi-Square   p-
                                                        value dof
                                                        Significance
Component
```

Created by data-to-paper (AI)

```
nova.osapi_compute.wsgi.server                                  -
            -   -            -
nova.compute.manager                                        3.042
    0.881   7           ns
nova.virt.libvirt.imagecache                                250.9  3.59
    e-52   5          ***
nova.api.openstack.compute.server_external_events               -
            -   -            -
nova.virt.libvirt.driver                                        -
            -   -            -
nova.compute.resource_tracker                                   -
            -   -            -
nova.metadata.wsgi.server                                       -
            -   -            -
nova.api.openstack.wsgi                                         -
            -   -            -
nova.compute.claims                                             -
            -   -            -
nova.scheduler.host_manager                                     -
            -   -            -
```

df_desc_stat.pkl

```
Level   INFO WARNING
EventId
E25      931       0
E34       82       0
E27       82       0
E35       82       0
E26       64       0
E21       44       0
E7        43       0
E36       30       0
E20       22       0
E24       22       0
E22       22       0
E9        22       0
E31       22       0
E15       22       0
E8        22       0
E12       22       0
E11       22       0
E37       21       0
E4        21       0
E5        21       0
```

df_eventid_level.pkl

```
                  Chi-Square   p-value dof Significance
```

Created by data-to-paper (AI)

```
EventId_Level_Test      133.8  7.51e-19  20          ***
```

additional_results.pkl

```
{
    'Total number of observations': 2000,
    'Chi-square statistic for EventId and Log Level': -,
    'p-value for EventId and Log Level': -,
    'dof for EventId and Log Level': -,
    'Number of components analyzed': 10,
}
```

D LaTeX Table Design

D.1 Code

The LaTeX Table Design was carried out using the following custom code:

```
# IMPORT
import pandas as pd
from my_utils import df_to_latex, df_to_figure, is_str_in_df,
    ↪ split_mapping, AbbrToNameDef

# PREPARATION FOR ALL TABLES AND FIGURES
all_mapping: AbbrToNameDef = {
    'Chi-Square': ('ChiSq', 'Chi-Square statistic for test of
        ↪ independence'),
    'p-value': ('PVal', 'Calculated probability for the Chi-Square
        ↪  statistic'),
    'dof': ('DF', 'Degrees of freedom for the Chi-Square test'),
    'Significance': ('Sig.', 'Statistical significance notation:
        ↪ *** p < 0.001, ** p < 0.01, * p < 0.05, ns p >= 0.05'),
    'INFO': ('INFO', 'Informational log level'),
    'WARNING': ('WARNING', 'Warning log level'),

    # Event definitions for glossary
    'E25': ('Event 25', 'Specific event pattern indexed as E25'),
    'E34': ('Event 34', 'Specific event pattern indexed as E34'),
    'E27': ('Event 27', 'Specific event pattern indexed as E27'),
    'E35': ('Event 35', 'Specific event pattern indexed as E35'),
    'E26': ('Event 26', 'Specific event pattern indexed as E26'),
    'E21': ('Event 21', 'Specific event pattern indexed as E21'),
    'E7': ('Event 7', 'Specific event pattern indexed as E7'),
    'E36': ('Event 36', 'Specific event pattern indexed as E36'),
    'E20': ('Event 20', 'Specific event pattern indexed as E20'),
    'E24': ('Event 24', 'Specific event pattern indexed as E24'),
    'E22': ('Event 22', 'Specific event pattern indexed as E22'),
    'E9': ('Event 9', 'Specific event pattern indexed as E9'),
```

Created by data-to-paper (AI)

```
    'E31': ('Event 31', 'Specific event pattern indexed as E31'),
    'E15': ('Event 15', 'Specific event pattern indexed as E15'),
    'E8': ('Event 8', 'Specific event pattern indexed as E8'),
    'E12': ('Event 12', 'Specific event pattern indexed as E12'),
    'E11': ('Event 11', 'Specific event pattern indexed as E11'),
    'E37': ('Event 37', 'Specific event pattern indexed as E37'),
    'E4': ('Event 4', 'Specific event pattern indexed as E4'),
    'E5': ('Event 5', 'Specific event pattern indexed as E5'),
    'EventId_Level_Test': ('EventId Level Test', 'Chi-Square test
        ↪ for association between EventId and Log Level')
}

## Process df_component_eventid_level:
df_component_eventid_level = pd.read_pickle('
    ↪ df_component_eventid_level.pkl')

# Not Applicable

# Rename rows and columns to make the table narrower
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_component_eventid_level, k))
mapping |= {
    'nova.osapi_compute.wsgi.server': ('Compute WSGI', 'Web Server
        ↪  Gateway Interface in OpenStack Compute Module'),
    'nova.compute.manager': ('Compute Manager', None),
    'nova.virt.libvirt.imagecache': ('Libvirt Image Cache', None),
    'nova.api.openstack.compute.server_external_events': ('Compute
        ↪  Server Ext. Events', None),
    'nova.virt.libvirt.driver': ('Libvirt Driver', None),
    'nova.compute.resource_tracker': ('Resource Tracker', None),
    'nova.metadata.wsgi.server': ('Metadata WSGI', 'Web Server
        ↪ Gateway Interface for Metadata Service in OpenStack'),
    'nova.api.openstack.wsgi': ('API WSGI', 'Web Server Gateway
        ↪ Interface for API in OpenStack'),
    'nova.compute.claims': ('Compute Claims', None),
    'nova.scheduler.host_manager': ('Scheduler Host Manager', None
        ↪ ),
}
abbrs_to_names, glossary = split_mapping(mapping)
df_component_eventid_level.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

df_to_latex(
    df_component_eventid_level, '
        ↪ df_component_eventid_level_formatted',
    caption="Chi-Squared Statistics for Association Between Event
        ↪ ID and Log Level Stratified by Component",
    glossary=glossary
)
```

Created by data-to-paper (AI)

```
## Process df_desc_stat:
df_desc_stat = pd.read_pickle('df_desc_stat.pkl')

# Not Applicable

# Since this is more suitable as a bar chart (frequency counts),
    ↪ we'll use df_to_figure
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_desc_stat, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_desc_stat.rename(columns=abbrs_to_names, index=abbrs_to_names,
    ↪ inplace=True)

df_to_figure(
    df_desc_stat, 'df_desc_stat_formatted',
    caption="Frequency of Top Event IDs Across Log Levels",
    glossary=glossary,
    kind='bar',
    y=['INFO', 'WARNING'],
    logy=True,
    xlabel='Event ID',
    ylabel='Frequency (Log Scale)'
)

## Process df_eventid_level:
df_eventid_level = pd.read_pickle('df_eventid_level.pkl')

# Not Applicable

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_eventid_level, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_eventid_level.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

df_to_latex(
    df_eventid_level, 'df_eventid_level_formatted',
    caption="Overall Chi-Squared Test for Event ID and Log Level
        ↪ Association",
    glossary=glossary
)
```

D.2 Provided Code

The code above is using the following provided functions:

Created by data-to-paper (AI)

```python
def df_to_latex(df,
        filename: str, caption: str,
        note: str = None,
        glossary: Dict[Any, str] = None,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    `note`: Note to be added below the table caption.
    `glossary`: Glossary for the table.
    """


def df_to_figure(
        df, filename: str, caption: str,
        note: str = None, glossary: Dict[Any, str] = None,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
        xlabel: str = None, ylabel: str = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.
    `note`: Note to be added below the figure caption.
    `glossary`: Glossary for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.
    `xlabel`: Label for the x-axis.
    `ylabel`: Label for the y-axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
```

Created by data-to-paper (AI)

```
        ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
        ↪ plotted above the error bars.

    If provided, the length of `y_ci`, and `y_p_value` should be
        ↪ the same as of `y`.

    Example:
    Suppose, we have:

    df_lin_reg_longevity = pd.DataFrame({
        'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
            ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
        'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
            ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
            ↪ [0.0001, ...],
    }, index=['var1', ...])

    then:
    df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
        ↪ caption='Coefficients of ...', kind='bar',
        y=['adjusted_coef', 'unadjusted_coef'],
        y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
        y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
    """


def is_str_in_df(df: pd.DataFrame, s: str):
    return any(s in level for level in getattr(df.index, 'levels',
        ↪  [df.index]) + getattr(df.columns, 'levels', [df.columns
        ↪ ]))

AbbrToNameDef = Dict[Any, Tuple[Optional[str], Optional[str]]]

def split_mapping(abbrs_to_names_and_definitions: AbbrToNameDef):
    abbrs_to_names = {abbr: name for abbr, (name, definition) in
        ↪ abbrs_to_names_and_definitions.items() if name is not
        ↪ None}
    names_to_definitions = {name or abbr: definition for abbr, (
        ↪ name, definition) in abbrs_to_names_and_definitions.
        ↪ items() if definition is not None}
    return abbrs_to_names, names_to_definitions
```

D.3 Code Output

df_component_eventid_level_formatted.pkl

```
\begin{table}[h]
```

Created by data-to-paper (AI)

```
\caption{Chi-Squared Statistics for Association Between Event ID
    and Log Level Stratified by Component}
\label{table:df-component-eventid-level-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lllll}
\toprule
 & ChiSq & PVal & DF & Sig. \\
Component &  &  &  &  \\
\midrule
\textbf{Compute WSGI} & - & - & - & - \\
\textbf{Compute Manager} & 3.042 & 0.881 & 7 & ns \\
\textbf{Libvirt Image Cache} & 250.9 & $<$1e-06 & 5 & *** \\
\textbf{Compute Server Ext. Events} & - & - & - & - \\
\textbf{Libvirt Driver} & - & - & - & - \\
\textbf{Resource Tracker} & - & - & - & - \\
\textbf{Metadata WSGI} & - & - & - & - \\
\textbf{API WSGI} & - & - & - & - \\
\textbf{Compute Claims} & - & - & - & - \\
\textbf{Scheduler Host Manager} & - & - & - & - \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item \textbf{ChiSq}: Chi-Square statistic for test of
    independence
\item \textbf{PVal}: Calculated probability for the Chi-Square
    statistic
\item \textbf{DF}: Degrees of freedom for the Chi-Square test
\item \textbf{Sig.}: Statistical significance notation: *** p $<$
    0.001, ** p $<$ 0.01, * p $<$ 0.05, ns p $>$= 0.05
\item \textbf{Compute WSGI}: Web Server Gateway Interface in
    OpenStack Compute Module
\item \textbf{Metadata WSGI}: Web Server Gateway Interface for
    Metadata Service in OpenStack
\item \textbf{API WSGI}: Web Server Gateway Interface for API in
    OpenStack
\end{tablenotes}
\end{threeparttable}
\end{table}
```

df_desc_stat_formatted.pkl

```
\begin{figure}[htbp]
\centering
\includegraphics{df_desc_stat_formatted.png}
\caption{Frequency of Top Event IDs Across Log Levels
INFO: Informational log level.
```

Created by data-to-paper (AI)

```
WARNING: Warning log level.
Event 25: Specific event pattern indexed as E25.
Event 34: Specific event pattern indexed as E34.
Event 27: Specific event pattern indexed as E27.
Event 35: Specific event pattern indexed as E35.
Event 26: Specific event pattern indexed as E26.
Event 21: Specific event pattern indexed as E21.
Event 7: Specific event pattern indexed as E7.
Event 36: Specific event pattern indexed as E36.
Event 20: Specific event pattern indexed as E20.
Event 24: Specific event pattern indexed as E24.
Event 22: Specific event pattern indexed as E22.
Event 9: Specific event pattern indexed as E9.
Event 31: Specific event pattern indexed as E31.
Event 15: Specific event pattern indexed as E15.
Event 8: Specific event pattern indexed as E8.
Event 12: Specific event pattern indexed as E12.
Event 11: Specific event pattern indexed as E11.
Event 37: Specific event pattern indexed as E37.
Event 4: Specific event pattern indexed as E4.
Event 5: Specific event pattern indexed as E5.}
\label{figure:df-desc-stat-formatted}
\end{figure}
% This latex figure presents "df_desc_stat_formatted.png",
% which was created from the df:
%
% index,"INFO","WARNING"
% "Event 25",931,0
% "Event 34",82,0
% "Event 27",82,0
% "Event 35",82,0
% "Event 26",64,0
% "Event 21",44,0
% "Event 7",43,0
% "Event 36",30,0
% "Event 20",22,0
% "Event 24",22,0
% "Event 22",22,0
% "Event 9",22,0
% "Event 31",22,0
% "Event 15",22,0
% "Event 8",22,0
% "Event 12",22,0
% "Event 11",22,0
% "Event 37",21,0
% "Event 4",21,0
% "Event 5",21,0
```

Created by data-to-paper (AI)

```
%
% To create the figure, this df was plotted with the command:
%
% df.plot(kind='bar', y=['INFO', 'WARNING'], logy=True, xlabel='
    Event ID', ylabel='Frequency (Log Scale)')
```

df_eventid_level_formatted.pkl

```
\begin{table}[h]
\caption{Overall Chi-Squared Test for Event ID and Log Level
    Association}
\label{table:df-eventid-level-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrlrl}
\toprule
 & ChiSq & PVal & DF & Sig. \\
\midrule
\textbf{EventId Level Test} & 133.8 & $<$1e-06 & 20 & *** \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item \textbf{ChiSq}: Chi-Square statistic for test of
    independence
\item \textbf{PVal}: Calculated probability for the Chi-Square
    statistic
\item \textbf{DF}: Degrees of freedom for the Chi-Square test
\item \textbf{Sig.}: Statistical significance notation: *** p $<$
    0.001, ** p $<$ 0.01, * p $<$ 0.05, ns p $>$= 0.05
\item \textbf{EventId Level Test}: Chi-Square test for association
     between EventId and Log Level
\end{tablenotes}
\end{threeparttable}
\end{table}
```